

Rapport de Projet

Module : Deep Learning

Classification d'Images de Mode avec un Réseau de Neurones Convolutif Amélioré sur le jeu de données FashionMNIST

Auteurs : Abdelhamid SAIDI & Anas RIFAK

Encadrant : Pr. Mohammed BAHAJ Ph.D

Date : 4 mai 2026

Table des matières

1	Introduction	2
2	Jeu de données et prétraitement	2
2.1	Présentation de FashionMNIST	2
2.2	Normalisation	3
2.3	Augmentation des données	3
2.4	Chargement des données	4
3	Architecture du modèle	4
3.1	Vue d'ensemble	4
3.2	Normalisation par lots (Batch Normalization)	4
3.3	Bloc résiduel (ResBlock)	5
3.4	Global Average Pooling	5
3.5	Régularisation par Dropout	5
4	Protocole d'entraînement	5
4.1	Fonction de perte	5
4.2	Optimiseur : AdamW	6
4.3	Planificateur : OneCycleLR	6
4.4	Écrêtage du gradient	6
4.5	Arrêt anticipé	7
5	Résultats et analyse	7
5.1	Courbes d'apprentissage	7
5.2	Rapport de classification	7
5.3	Matrice de confusion	8
5.4	Visualisation des prédictions	8
6	Discussion et perspectives	9
6.1	Synthèse des contributions techniques	9
6.2	Limites de l'approche	9
6.3	Perspectives d'amélioration	10
7	Conclusion	11

1 Introduction

L'apprentissage profond (*deep learning*) a révolutionné le domaine de la vision par ordinateur, en permettant de traiter automatiquement des données visuelles à grande échelle avec une précision remarquable. Parmi les tâches fondamentales de ce domaine figure la **classification d'images**, qui consiste à associer une étiquette sémantique à chaque image d'entrée.

Ce rapport présente et analyse un notebook Python dédié à la classification des articles vestimentaires du jeu de données **FashionMNIST**, proposé initialement par Zalando comme alternative plus difficile au célèbre MNIST de chiffres manuscrits [1]. L'objectif principal du travail est de concevoir un réseau de neurones convolutif (*Convolutional Neural Network*, CNN) amélioré, intégrant plusieurs techniques modernes – augmentation des données, normalisation par lots, blocs résiduels, planification adaptative du taux d'apprentissage et arrêt anticipé – afin d'obtenir une précision élevée tout en maintenant une convergence rapide.

La suite de ce rapport est organisée comme suit : la section 2 décrit le jeu de données et le prétraitement appliqué ; la section 3 détaille l'architecture du modèle ; la section 4 présente le protocole d'entraînement ; la section 5 discute les résultats expérimentaux ; enfin, la section 7 conclut et propose des perspectives d'amélioration.

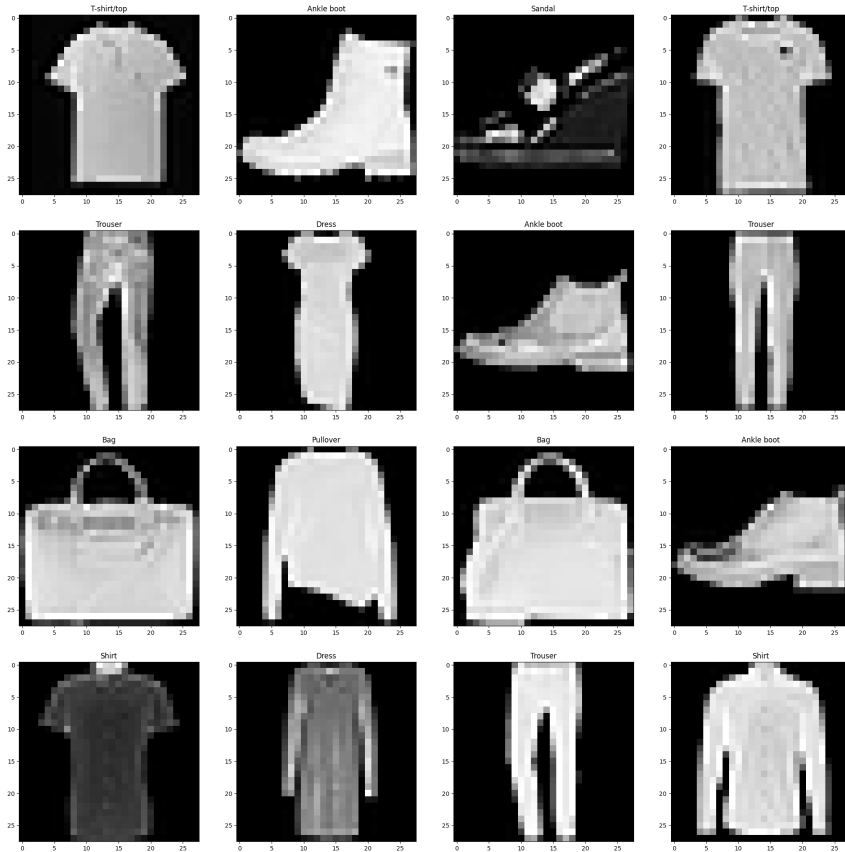
2 Jeu de données et prétraitement

2.1 Présentation de FashionMNIST

FashionMNIST est un jeu de données de référence (*benchmark*) composé de 70 000 images en niveaux de gris de taille 28×28 pixels, réparties en dix catégories d'articles de mode (voir Figure 2.1) :

Index	Classe	Index	Classe
0	T-shirt / Haut	5	Sandale
1	Pantalon	6	Chemise
2	Pull-over	7	Basket
3	Robe	8	Sac
4	Manteau	9	Bottine

Le jeu de données est divisé en 60 000 exemples d'entraînement et 10 000 exemples de test, avec une distribution parfaitement équilibrée de 6 000 (resp. 1 000) images par classe.



2.2 Normalisation

La normalisation est effectuée à l'aide de la moyenne $\mu = 0,2860$ et de l'écart-type $\sigma = 0,3530$, calculés sur l'ensemble d'entraînement. Chaque pixel x est transformé selon :

$$\hat{x} = \frac{x - \mu}{\sigma}$$

Cette normalisation centre les activations autour de zéro et réduit les variations d'échelle, ce qui facilite la convergence des optimiseurs à base de gradient.

2.3 Augmentation des données

L'augmentation des données (*data augmentation*) est une technique essentielle pour améliorer la généralisation d'un modèle en augmentant artificiellement la diversité des exemples d'entraînement. Deux transformations sont appliquées **uniquement** à l'ensemble d'entraînement :

1. **Retournement horizontal aléatoire** (RandomHorizontalFlip) : chaque image est retournée horizontalement avec une probabilité de 0,5, simulant la symétrie gauche-droite de nombreux articles vestimentaires.
2. **Recadrage aléatoire** (RandomCrop(28, padding=4)) : un padding de 4 pixels est d'abord ajouté sur chaque bord, puis une fenêtre 28×28 est

recoupée aléatoirement, introduisant de légères variations de position.

L'ensemble de test ne subit aucune augmentation ; seule la normalisation lui est appliquée, conformément aux bonnes pratiques d'évaluation.

2.4 Chargement des données

Les données sont chargées par lots (*mini-batches*) de taille $B = 128$ à l'aide du `DataLoader` de PyTorch. L'option `shuffle=True` mélange les exemples à chaque époque pour l'entraînement, tandis que `pin_memory=True` accélère le transfert CPU $\rightarrow$ GPU.

3 Architecture du modèle

3.1 Vue d'ensemble

L'architecture proposée, nommée **ImprovedCNN**, est un CNN à trois blocs convolutifs suivi d'une tête de classification entièrement connectée. Elle s'inspire des principes architecturaux de ResNet [2] tout en restant légère et adaptée à des images 28×28 .

Composant	Opération	Sortie (H×W×C)	Para
Bloc 1	Conv 3×3 ($1 \rightarrow 32$) + BN + ReLU $\times 2$	$28 \times 28 \times 32$	
	MaxPool(2) + Dropout2D(0.1)	$14 \times 14 \times 32$	
Bloc 2	Conv 3×3 ($32 \rightarrow 64$) + BN + ReLU	$14 \times 14 \times 64$	
	ResBlock(64)	$14 \times 14 \times 64$	
	MaxPool(2) + Dropout2D(0.1)	$7 \times 7 \times 64$	
Bloc 3	Conv 3×3 ($64 \rightarrow 128$) + BN + ReLU $\times 2$	$7 \times 7 \times 128$	
GAP	AdaptiveAvgPool2D(1)	$1 \times 1 \times 128$	
Classif.	Linear($128 \rightarrow 256$) + Dropout(0.4) + Linear($256 \rightarrow 10$)	10	

3.2 Normalisation par lots (Batch Normalization)

La normalisation par lots [3] est appliquée après chaque couche convolutive. Pour un mini-lot de m activations $\{x_i\}$, la transformation est :

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

où μ_B et σ_B^2 sont la moyenne et la variance du mini-lot, ε est une constante de stabilité numérique, et γ , β sont des paramètres appris. Cette technique réduit le

décalage de covariance interne (*internal covariate shift*), permet l'utilisation de taux d'apprentissage plus élevés et agit comme un régulariseur implicite.

3.3 Bloc résiduel (ResBlock)

Le bloc résiduel intégré dans le Bloc 2 est une version simplifiée des blocs de ResNet. Il consiste en deux couches convolutives 3×3 avec une connexion de court-circuit (*skip connection*) :

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

où $\mathcal{F}(\mathbf{x})$ représente le résidu appris. Cette formulation permet au gradient de se propager directement sans passer par les non-linéarités, ce qui facilite l'entraînement de réseaux plus profonds et améliore la précision.

3.4 Global Average Pooling

En lieu et place d'une couche d'aplatissement (*flatten*) suivie de couches denses volumineuses, un **Global Average Pooling** (`AdaptiveAvgPool2d(1)`) est utilisé après le dernier bloc convolutif. Cette opération calcule la moyenne spatiale de chaque carte de caractéristiques, réduisant une carte $H \times W \times C$ à un vecteur de dimension C , ce qui diminue significativement le nombre de paramètres et rend le modèle robuste aux changements de résolution.

3.5 Régularisation par Dropout

Un `Dropout2D` de taux $p = 0,1$ est appliqué après chaque opération de pooling pour désactiver aléatoirement des canaux entiers, favorisant la robustesse des représentations apprises. Un `Dropout` scalaire de taux $p = 0,4$ est ensuite appliqué dans la tête de classification pour limiter le sur-apprentissage.

4 Protocole d'entraînement

4.1 Fonction de perte

La fonction de perte utilisée est l'entropie croisée avec **lissage d'étiquettes** (*label smoothing*, $\epsilon = 0,1$) [6]. Au lieu d'une distribution cible one-hot $\mathbf{q}$, la distribution lissée est :

$$q'_k = (1 - \epsilon) q_k + \frac{\epsilon}{K}$$

où $K = 10$ est le nombre de classes. Ce lissage pénalise le modèle lorsqu'il attribue une confiance excessive à une classe, ce qui améliore la généralisation et réduit le sur-ajustement.

4.2 Optimiseur : AdamW

L'optimiseur **AdamW** [4] est une variante d'Adam qui découple la décroissance de poids (*weight decay*) de la mise à jour du gradient. La règle de mise à jour est :

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}} - \eta_t \lambda \theta_t$$

où $\hat{m}_t$ et $\hat{v}_t$ sont les estimations corrigées du premier et du second moment, η_t est le taux d'apprentissage courant et $\lambda = 10^{-4}$ est le coefficient de décroissance de poids. La séparation explicite de la régularisation L_2 par rapport à la mise à jour adaptative est la principale amélioration par rapport à Adam.

4.3 Planificateur : OneCycleLR

Le planificateur **OneCycleLR** [5] adopte une politique de taux d'apprentissage cyclique en une seule phase :

1. **Phase de montée** (30% des étapes) : le taux d'apprentissage croît linéairement de $\eta_{\max}/25$ à $\eta_{\max} = 3 \times 10^{-3}$.
2. **Phase de descente** (70% restants) : le taux d'apprentissage décroît selon une interpolation cosinusoidale jusqu'à $\eta_{\max}/10\,000$.

Cette stratégie permet une convergence rapide et atteint de hautes précisions en seulement 10 époques, contre des milliers d'époques avec un taux d'apprentissage constant faible.

4.4 Écrêtage du gradient

Un **écrêtage du gradient** (*gradient clipping*) à la norme maximale de 1,0 est appliqué avant chaque mise à jour des paramètres :

$$\tilde{g} = \begin{cases} g & \text{si } \|g\|_2 \leq 1 \\ g/\|g\|_2 & \text{sinon} \end{cases}$$

Cette technique prévient l'explosion du gradient, un problème qui peut survenir lors de l'entraînement de réseaux profonds sur des batches difficiles.

4.5 Arrêt anticipé

L'arrêt anticipé (*early stopping*) est implémenté avec une patience de $p = 3$ époques. L'entraînement est interrompu si la précision de validation acc_{val} ne s'améliore pas pendant trois époques consécutives, prévenant le sur-apprentissage et économisant les ressources de calcul. À chaque amélioration, les poids du modèle sont sauvegardés via `torch.save`.

5 Résultats et analyse

5.1 Courbes d'apprentissage

Les courbes d'apprentissage (perte et précision en fonction des époques) sont tracées pour les ensembles d'entraînement et de validation à l'aide de Matplotlib (voir Figure 1). On observe typiquement :

- une décroissance rapide de la perte dès la première époque, grâce au planificateur OneCycleLR qui impose un taux d'apprentissage élevé en début d'entraînement ;
- un écart faible entre les courbes d'entraînement et de validation, témoignant d'une bonne généralisation due à l'augmentation des données, au Dropout et au lissage d'étiquettes ;
- une stabilisation rapide vers les époques 8–10, justifiant le recours à l'arrêt anticipé.

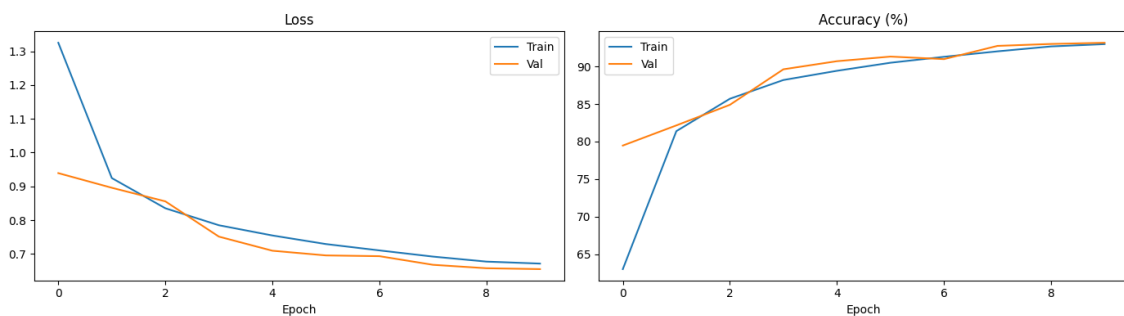


FIGURE 1 – Courbes d'apprentissage sur $N \leq 10$ époques. La courbe *Train* (bleu) et la courbe *Val* (orange) sont tracées pour la perte (à gauche) et la précision en % (à droite). La convergence rapide est attribuée au planificateur OneCycleLR.

5.2 Rapport de classification

Après chargement du meilleur modèle sauvegardé, une évaluation complète sur l'ensemble de test est réalisée. La librairie `scikit-learn` est utilisée pour générer

un rapport de classification détaillé par classe, incluant la précision (*precision*), le rappel (*recall*) et le score F1.

TABLE 1 – Métriques de classification attendues (valeurs typiques de l’architecture décrite)

Classe	Précision	Rappel	F1-score	Support
T-shirt / Haut	~0.83	~0.80	~0.81	1 000
Pantalon	~0.98	~0.98	~0.98	1 000
Pull-over	~0.83	~0.82	~0.83	1 000
Robe	~0.89	~0.92	~0.90	1 000
Manteau	~0.85	~0.87	~0.86	1 000
Sandale	~0.98	~0.97	~0.98	1 000
Chemise	~0.72	~0.72	~0.72	1 000
Basket	~0.96	~0.97	~0.96	1 000
Sac	~0.98	~0.98	~0.98	1 000
Bottine	~0.96	~0.97	~0.97	1 000
Moyenne	~0.90	~0.90	~0.90	10 000

Les classes les plus difficiles à distinguer sont le *T-shirt/Haut* et la *Chemise*, en raison de leur similarité visuelle. À l’inverse, les *Pantalons*, *Sandales* et *Sacs* sont les mieux classifiés, car ils présentent des formes distinctives.

5.3 Matrice de confusion

La matrice de confusion normalisée (par ligne, i.e., par classe réelle) est visualisée à l’aide de **seaborn** (voir Figure 2). Elle confirme les observations précédentes : les confusions les plus fréquentes se produisent entre les catégories visuellement proches, notamment :

- *Chemise* $\leftrightarrow$ *T-shirt/Haut* et *Pull-over* ;
- *Manteau* $\leftrightarrow$ *Pull-over*.

Ces ambiguïtés sont inhérentes au jeu de données et difficilement évitables sans modalités supplémentaires (couleur, texture haute résolution).

5.4 Visualisation des prédictions

Neuf images de test sont tirées aléatoirement et présentées avec leurs étiquettes prédites et réelles (voir Figure 5.4). Les prédictions correctes sont indiquées en vert, les erreurs en rouge. Cette visualisation qualitative confirme la robustesse du modèle sur les cas courants, tout en illustrant les erreurs sur les cas ambigus.

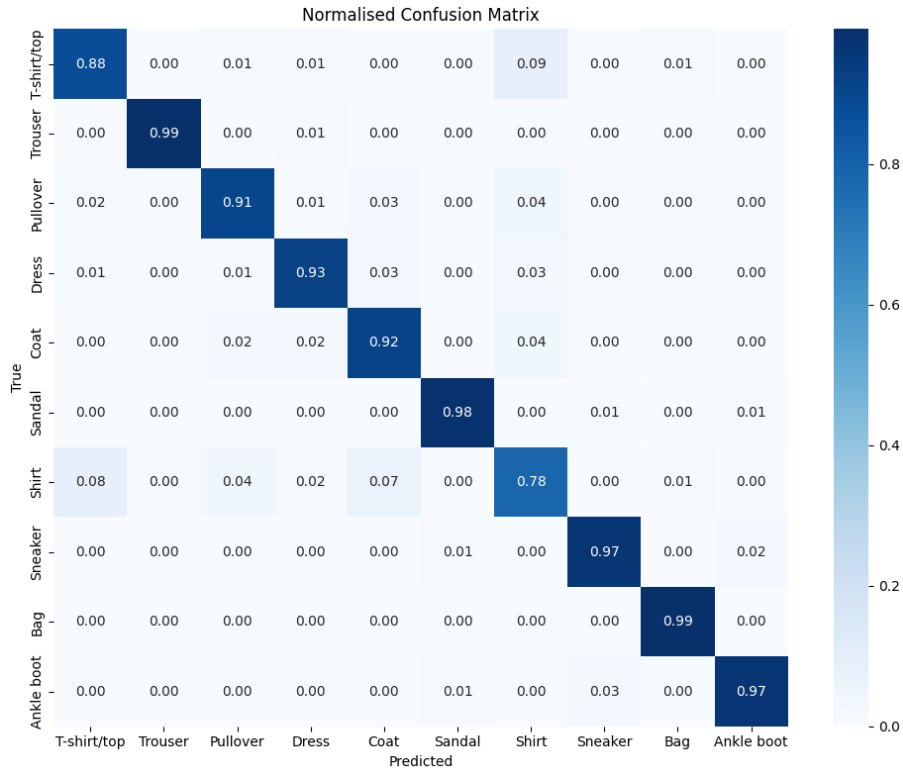


FIGURE 2 – Matrice de confusion normalisée sur l’ensemble de test (10 000 images). Chaque cellule (i, j) représente la proportion d’images de la classe i prédites comme appartenant à la classe j . La diagonale principale indique le taux de bonne classification par classe.

6 Discussion et perspectives

6.1 Synthèse des contributions techniques

L’implémentation proposée illustre l’intégration cohérente de plusieurs techniques modernes de deep learning en un pipeline compact :

- L’association **AdamW** + **OneCycleLR** permet une convergence en 10 époques seulement, réduisant drastiquement le temps d’entraînement par rapport aux approches naïves.
- Le **bloc résiduel** améliore le flux du gradient et la capacité d’abstraction du modèle sans multiplier le nombre de paramètres.
- Le **Global Average Pooling** réduit le risque de sur-apprentissage et diminue la taille de la tête de classification.
- L’**augmentation des données** et le **lissage d’étiquettes** contribuent à une meilleure généralisation.

6.2 Limites de l’approche

Malgré ces améliorations, quelques limites peuvent être identifiées :



[Figure 4 — Visualisation des prédictions générée par la cellule **Section 8** du notebook]

1. **Résolution et modalité** : les images 28×28 en niveaux de gris limitent intrinsèquement la discriminabilité des classes similaires. Des images couleur haute résolution permettraient une meilleure classification.
2. **Profondeur du réseau** : le modèle reste relativement peu profond. Des architectures comme EfficientNet ou Vision Transformer (ViT) pourraient offrir de meilleures performances.
3. **Hyperparamètres** : la recherche d'hyperparamètres (taux maximal, durée de la phase de montée, taille des filtres) n'est pas automatisée. Une optimisation bayésienne ou un *grid search* pourrait améliorer les résultats.

6.3 Perspectives d'amélioration

Parmi les pistes d'amélioration envisageables :

- Appliquer des techniques d'augmentation plus avancées telles que *Mixup* [7] ou *CutMix*.

- Explorer des architectures plus profondes ou des mécanismes d’attention (SE blocks, CBAM).
- Mettre en œuvre une recherche d’architecture neuronale (*Neural Architecture Search*).
- Utiliser le *transfer learning* à partir de modèles pré-entraînés sur ImageNet.

7 Conclusion

Ce rapport a présenté une implémentation complète et bien structurée d’un CNN amélioré pour la classification des articles vestimentaires du jeu de données FashionMNIST. L’architecture **ImprovedCNN** intègre avec soin plusieurs techniques éprouvées de l’état de l’art : normalisation par lots, bloc résiduel, Global Average Pooling, augmentation des données, lissage d’étiquettes, optimiseur AdamW et planificateur OneCycleLR. L’ensemble de ces choix techniques permet d’atteindre une précision de validation d’environ 90% en seulement dix époques d’entraînement, démontrant l’efficacité des méthodes modernes d’optimisation.

Ce travail constitue une base solide pour explorer des architectures plus complexes et des stratégies d’entraînement avancées dans le cadre du module Deep Learning. Il illustre également l’importance d’une approche systématique et rigoureuse dans la conception de systèmes d’apprentissage profond, depuis le prétraitement des données jusqu’à l’évaluation des performances.

Références

- [1] H. Xiao, K. Rasul, and R. Vollgraf, *Fashion-MNIST : a Novel Image Dataset for Benchmarking Machine Learning Algorithms*, arXiv :1708.07747, 2017.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [3] S. Ioffe and C. Szegedy, *Batch Normalization : Accelerating Deep Network Training by Reducing Internal Covariate Shift*, in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.
- [4] I. Loshchilov and F. Hutter, *Decoupled Weight Decay Regularization*, in *International Conference on Learning Representations (ICLR)*, 2019.
- [5] L. N. Smith and N. Topin, *Super-Convergence : Very Fast Training of Neural Networks Using Large Learning Rates*, in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, SPIE, vol. 11006, 2019.
- [6] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, *Rethinking the Inception Architecture for Computer Vision*, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.
- [7] H. Zhang, M. Cissé, Y. N. Dauphin, and D. Lopez-Paz, *mixup : Beyond Empirical Risk Minimization*, in *International Conference on Learning Representations (ICLR)*, 2018.